REFERENZ

Datenstruktur und Verarbeitungseinheiten

Flags, Register, Maschinenwort und Befehlswortaufbau (Bitfelder) sowie die Zahlendarstellungen des C 8205

Dieses PDF gibt die Quelle im Wortlaut wieder. Die Quelle hat keine Markdown-Überschriften; das Skript setzt Zwischenüberschriften aus ihren Stichwortzeilen und die beiden Registerlisten als Tabellen. Der Text selbst bleibt unverändert, weich umbrochene Zeilen sind zu Absätzen zusammengeführt.

Quelle	`C8205-Interpreter-Datenstruktur_und_Verarbeitungseinheiten.md`	0.95-004
Stand	11.11.2026	
Umfang	7 Seiten, A4 hoch	
Erzeugt mit	`erzeuge_datenstruktur_pdf.py`	

> Maßgeblich bleibt die Quelldatei. Dieses PDF wird nicht automatisch nachgeführt: Es ist aktuell, solange die Version oben mit der Versionszeile der Quelldatei übereinstimmt.

Inhaltsverzeichnis

Namensbereiche und Zahlensysteme . . . 3
Übersicht der Zahlendarstellungen . . . 3
Dualzahlen . . . 3
Ganze Festkommazahlen (Programmieranleitung 6.2) . . . 3
Gleitkommazahlen, interne Darstellung . . . 4
Flags und Register . . . 5
Flags . . . 5
Register und Begriffe des Befehlsablaufs . . . 6
Maschinenwort . . . 7
Befehlswort . . . 7

C 8205 – Datenstruktur und Verarbeitungseinheiten

Quelle: C8205-Interpreter-Datenstruktur_und_Verarbeitungseinheiten.md · im Wortlaut der Quelle

Version: 0.95-004

Namensbereiche und Zahlensysteme

Namensbereiche: Flags, Register, Felder des Befehlswortes sowie die Knoten und Konnektoren der Knotentabelle sind getrennte Beschreibungsbereiche. Innerhalb eines Bereichs muss jeder Name eindeutig sein, zwischen Bereichen dürfen gleiche Namen vorkommen (z.B. Flag A und Feld A, Feld B und Konnektor B) und sind voneinander unabhängig. Wird ein Name außerhalb seines Bereichs benutzt, wird er qualifiziert: "Flag A", "Feld A". <> gelten an Stelle einer Qualifizierung, da sie nur bei Registern und Speicherinhalten vorkommen.

Zahlensysteme: Original-Zahlen in Befehlen, Registern und im Speicher sind oktal (außer ausdrücklich als dual oder dezimal gekennzeichnet). Element-Referenznummern dieses Dokuments sind immer dezimal.

Übersicht der Zahlendarstellungen

Dieses Dokument ist der Einstieg für alle Zahlendarstellungen des Projekts. Der Rechner selbst kennt nur die Dualzahl (Addition und Subtraktion sind verdrahtet); alle übrigen Darstellungen sind Konventionen von Programmen. Die Tabelle zeigt, wo jede Darstellung definiert ist.

Darstellung	Art	Definiert in
Dualzahl (Festkomma, gebrochen; Zweierkomplement)	Hardware: verdrahtete Addition und Subtraktion	dieses Dokument, Abschnitt „Dualzahlen“
Ganze Festkommazahl (Komma hinter z_{32})	Programmkonvention des Rechners (Programmieranleitung 6.2), unabhängig vom BIS	dieses Dokument, Abschnitt „Ganze Festkommazahlen“
Gleitkommazahl, intern (Basis 2, e′ = e + 32)	Programmkonvention des Rechners (Programmieranleitung 6.3), unabhängig vom BIS; das BIS übernimmt sie	dieses Dokument, Abschnitt „Gleitkommazahlen, interne Darstellung“
Gleitkommazahl, extern (Basis 10), Dezimalschreibweise	Konvention des BIS (Ein- und Ausgabe)	`C8205-Interpreter-BIS.md`, Abschnitt BIS-1.1
Tetradendarstellung	Konvention des BIS (Ein- und Ausgabe)	`C8205-Interpreter-BIS.md`, Abschnitt BIS-1.1
Zahlenbereiche der Eingabe je Kennzeichen K_1	Konvention des BIS	`C8205-Interpreter-BIS.md`, Abschnitt BIS-2.2.6

Dualzahlen

- Bei der Ausführung der verdrahteten Rechenbefehle (Addition, Subtraktion) wird das interne Wort vom Rechner als **Dualzahl in Festkommadarstellung** interpretiert.
- Die Stellung des Kommas wird nicht festgelegt; man denkt es sich beispielsweise zwischen z_0 und z_1.
- **z_0** enthält das **Vorzeichen**.
- **z_{32}** ist das **niederwertigste Bit** (z_1 demnach das höchstwertige Betragsbit, unmittelbar nach dem Vorzeichenbit), bei vorzeichenlosen Zahlen ist z_0 das höchstwertige Bit.
- Positive Zahlen z: $z = \Sigma\ (i=1 \text{ bis } 32)\ z_i \cdot 2^{-i}$
- Negative Zahlen $\tilde{z}$ werden als **Zweierkomplement** dargestellt: $\tilde{z} = 2 - z$. Dabei werden alle Bits vor dem niedrigstwertigen mit 1 besetzten Bit negiert (siehe Original-Abb. 7, hier nicht enthalten).

Ganze Festkommazahlen (Programmieranleitung 6.2)

- Steht das Komma hinter z_{32}, wird das Wort als **ganze Zahl** interpretiert. Addition und Subtraktion sind dieselben verdrahteten Befehle wie bei der gebrochenen Festkommazahl; nur die Deutung des Wortes ändert sich.
- Positive Zahlen: $z = \Sigma\ (i=1 \text{ bis } 32)\ z_i \cdot 2\text{^}(32-i)$, $z_0 = 0$. Negative Zahlen: Zweierkomplement $\tilde{z} = 2^{33} - z$, $z_0 = 1$. Die Null gilt als positiv.
- Zahlenbereich: $-2^{32} \le z \le 2^{32} - 1$. Die kleinste positive Zahl ist die Eins (Wort `00000000001`). Das ist der Bereich des Maschinenwortes; die Eingabe unter BIS ist enger (`C8205-Interpreter-BIS.md`, Abschnitt BIS-1.1).

- Multiplikation und Division (anders als Addition und Subtraktion) sind nicht verdrahtet, sondern werden durch Programme mit Gruppenbefehlen gebildet; die Stellung des Kommas legt dort die Programmierung fest (gebrochen: Programmieranleitung 6.1, ganz: 6.2). Testaufträge: `Tests/bsp_6_1_*.txt`, `Tests/bsp_6_2_*.txt`.
- Der Assembler `c82ass` schreibt eine gebrochene Festkommazahl (Komma zwischen z_0 und z_1) mit `dwf` aus einem Dezimalbruch (ab c82ass 1.01-004; Assemblerdokumentation 3.3.20): `dwf 0.5` ergibt `20000000000`, `dwf -0.5` `60000000000`; Bereich $-1 \le x \le 1 - 2^{-32}$.

Gleitkommazahlen, interne Darstellung

(Programmieranleitung 6.3, Abb. 28; ergänzt 07.10.2026 um die Definitionen des Bausteininterpretiersystems BIS; die externe Darstellung steht in `C8205-Interpreter-BIS.md`, Abschnitt BIS-1.1)

- **Allgemeine halblogarithmische Schreibweise:** Jede Zahl lässt sich als Produkt aus Mantisse m und Basispotenz mit dem Exponenten e darstellen: $z = \pm m \cdot B^e$ (B = Basis des Zahlensystems). Damit die Darstellung eindeutig wird, gilt die Normalisierungsvorschrift $B^{-1} \le m < 1$ (für $z \ne 0$). Die Mantisse m bestimmt die Ziffernfolge von z, der Exponent e legt fest, an welcher Stelle der Mantisse das Komma steht.
- **Intern** (Basis B = 2, Definition des BIS): $z = \pm m \cdot 2^e$ mit der **normalisierten Mantisse** $0{,}5 \le |m| < 1$ für $z \ne 0$ (dezimal betrachtet). Exponent, Betrag der Mantisse und Vorzeichen werden getrennt gespeichert. Die Gleitkommadarstellung ist reine Programmkonvention: Der Rechner kennt keine Gleitkommabefehle, das Programm interpretiert das Maschinenwort. Die externe Darstellung (Ein- und Ausgabe) und die Tetradendarstellung sind Konventionen des BIS (`C8205-Interpreter-BIS.md`, Abschnitt BIS-1.1).
- Aufteilung des Maschinenwortes:

Bits	Inhalt	Breite
z_0	Vorzeichen der Mantisse (0 = +, 1 = −), Vorzeichen-Betrags-Darstellung	1 Bit
z_1 – z_6	Maschinenexponent $e' = e + 32 = \Sigma$ (i=0 bis 5) $e_i \cdot 2^i$, höchstwertiges Bit $e_5 = z_1$, niederwertigstes $e_0 = z_6$	6 Bit
z_7 – z_{32}	Mantissenbetrag $m_1\ m_2 \ldots m_{26}$ ($m_1 = z_7$, $m_{26} = z_{32}$)	26 Bit

Stellenwerte (Abbildung der BIS-Dokumentation): $z_1 \ldots z_6$ tragen die Gewichte $2^5, 2^4, 2^3, 2^2, 2^1, 2^0$ (Exponent e′); $z_7 \ldots z_{32}$ tragen die Gewichte $2^{-1}, 2^{-2}, 2^{-3}, \ldots, 2^{-24}, 2^{-25}, 2^{-26}$ (Mantisse m); z_0 ist das Vorzeichen (Vz).

- Vorzeichenstelle: für $z \ge 0$ ist $z_0 = 0$, für $z < 0$ ist $z_0 = 1$.
- Um Vorzeichenrechnungen im Exponenten zu vermeiden, wird der tatsächliche Exponent intern um $2^5 = 32$ erhöht angegeben: $e' = e + 32$.
- Die Mantisse ist **nicht** im Zweierkomplement abgelegt, sondern als Betrag mit getrenntem Vorzeichen z_0. Wegen der Normalisierung ist $m_1 = 1$: Bei allen Gleitkommazahlen außer der Gleitkommanull ist das Bit z_7 stets mit Eins besetzt.
- Exponentenbereich $0 \le e' \le 63$, also $-32 \le e \le 31$. Mantissenbereich $0{,}5 \le |m| \le 1 - 2^{-26}$. Zahlenbereich: $2^{-33} = 0{,}5 \cdot 2^{-32} \le |z| \le (1 - 2^{-26}) \cdot 2^{31} = 2^{31} - 2^5$, sowie $z = 0$. Die kleinste darstellbare positive Zahl $\ne 0$ ist $z = 0{,}5 \cdot 2^{-32} = 2^{-33}$.
- **Null:** Die Gleitkommanull entspricht der dualen Null, d. h. alle Dualstellen sind mit Null besetzt. Eindeutig mit $m_i = 0$ (i = 1 ... 26) und $e = -32$, d. h. $e' = 0$. Das ist das Wort `00000000000` (positive Null, $z_0 = 0$). Die Bitkombination „negative Null" ($z_0 = 1$, Rest 0) ist keine gültige Darstellung.
- Beispiel: $z = -16 = -0{,}5 \cdot 2^5$ ist `1 100101 10000000000000000000000000` (Vorzeichen, Exponent $e' = 37$, Mantissenbetrag), oktal `62600000000`. Zum Vergleich $z = +16$ ist `22600000000`.
- Der Assembler `c82ass` schreibt Gleitkommawörter mit `dwg` (ab c82ass 1.01-004; Assemblerdokumentation 3.3.21): `dwg -16` ergibt `62600000000`, `dwg 1.5E3` das Wort für 1500, `dwg 0` die Null (nie die „negative Null“). Er rechnet den Dezimalwert exakt um und rundet die Mantisse auf 26 Bit (genau in der Mitte weg von 0); Zahlenbereich $2^{-33} \le |z| \le 2^{31} - 2^5$ und 0. Ein Gleitkommawort lässt sich auch weiter als Oktalzahl mit führender 0 mit `dw` angeben (z. B. −16: `dw 062600000000`, wie `dw 004000000000` in den Beispielprogrammen).
- Folgerungen aus dem Aufbau (genutzt in den Beispielprogrammen 6.3.1 bis 6.3.3):
  - **Betrag:** Linksverschiebung (V^1) wirft z_0 heraus, anschließende Rechtsverschiebung setzt $z_0 = 0$ ein: $|x|$.
  - **Vorzeichenumkehr:** Addition der Konstante `40000000000` (nur z_0 gesetzt) kehrt z_0 um. Bei $x < 0$ entsteht dabei ein Überlauf (der Übertrag verlässt die Zahl), der durch einen nachfolgenden Überlaufsprung gelöscht werden muss, sonst hält der Rechner an. Die Null darf nicht umgekehrt werden (sonst „negative Null"); sie wird vorher mit dem Vergleichersprung 3 abgefangen.

- **Betragsvergleich:** Weil der Exponent vor der Mantisse steht und beide bei normalisierten Zahlen mit $z_0 = 0$ positiv geordnet sind, ordnen die Betragswörter ($z_0 = 0$) wie die ganzen Zahlen: |a| < |b| genau dann, wenn das Wort von |a| als ganze Zahl kleiner ist. Der markierte Einzelbefehl (Vergleicher setzen) genügt.
- Testaufträge: `Tests/bsp_6_3_1_*.txt` (Betrag), `bsp_6_3_2_*.txt` (Vorzeichenumkehr), `bsp_6_3_3_*.txt` (Ordnen).

Die externe Gleitkommadarstellung (Dezimalschreibweise, Eingabezahlenbereich, Genauigkeit) und die Tetradendarstellung sind Konventionen des BIS und stehen in `C8205-Interpreter-BIS.md`, Abschnitt BIS-1.1.

Flags und Register

Zahlen treten auf

- in Flags (1 dual-bit)
- in Registern (oktal oder dual)
- im Trommelspeicher (oktal oder dual)

Flags

Der Rechner selbst ist Hardware. Er wird vom Menschen bedient und arbeitet Befehle sukzessive ab (das Programm). 1968 gab es dafür keine Bildschirme mit anklickbaren Schaltflächen; man benutzte Schalter, die räumlich konzentriert am sogenannten Bedienpult angeordnet waren. Im Bedienpult befinden sich Lampen, die den aktuellen Zustand aller Flags anzeigen, sowie Schalter, mit denen ein Mensch diese Flags setzen oder löschen kann. Der Rechner reagiert als elektronische Hardware auf den Zustand der Flags; ein Mensch kann den Rechner also über die Schalter beeinflussen. Beispiel: Wird der Schalter SP gesetzt, kann der Rechner das auswerten, wenn ein entsprechender Befehl es abtestet. Dass SP gesetzt ist, wird im Flag SP gespeichert und in der Lampe SP angezeigt. Schalter und Flag desselben Namens (z.B. SP) sind also keine zwei verschiedenen Dinge, sondern zwei Seiten derselben Sache: der Schalter ist die Handlung, das Flag der gespeicherte Zustand, die Lampe seine Anzeige. Ausnahme ist BS: Der Schalter BS ist nur die Bedingung des Befehls "Bedingter Stopp"; das Flag BS wird erst gesetzt, wenn dieser Befehl bei eingerastetem Schalter tatsächlich stoppt (s. Flag-Tabelle).

Ein Flag ist ein elektronischer Speicher für den Wert 0 oder 1. Ein Flag setzt seinen Wert nicht selbst, sondern wird vom Rechenwerk (RW) oder vom Menschen mittels Schalter mit 0 oder 1 belegt; wer zuletzt setzt, bestimmt den Zustand. Den meisten Flags ist im Bedienpult eine Lampe zugeordnet, damit Menschen sie erkennen können. Befehle reagieren bei ihrer Abarbeitung im RW auf den Wert bestimmter Flags oder setzen bestimmte Flags neu.

Flags sind:

Flag-Name	Beschreibung
E	=1, wenn ein Eingabebefehl aufgestellt ist. Zur Eingabe ist zusätzlich die Anwahl eines Eingabegerätes erforderlich. E erlischt nach der Eingabe eines Zeichens mit der Umstellung des Eingabebefehls; E erlischt nicht, wenn der Eingabebefehl nicht ausgeführt wird (z. B. fehlende Anwahl), und leuchtet bei erneutem Eingabebefehl sofort wieder (Bedienungsanleitung 2.3.2)
A	=1, wenn ein Ausgabebefehl aufgestellt ist. Zur Ausgabe ist zusätzlich die Anwahl eines Ausgabegerätes erforderlich. A erlischt mit der Umstellung des Ausgabebefehls (nach der Übergabe des Zeichens an den 1-Zeichen-Puffer, nicht erst nach Abschluss der Geräteausgabe) und leuchtet bei erneutem Ausgabebefehl sofort wieder. Fehlt die benötigte Anwahl oder ist die programmierte Anwahl nicht existent, stoppt der Rechner mit gesetztem Flag A (Lampe A) (Bedienungsanleitung 2.3.2, Originaldokumentation 4.3.2.5)
T	Subflag, ist =1 für "Tastatur aktiv" im Kanal K0; T und SW sind ALTERNATIV
SW	Subflag, ist =1 für "Schreibwerk (SW) aktiv" im Kanal K0; T und SW sind ALTERNATIV
K1	=1, wenn Kanal 1 angewählt ist
K2	=1, wenn Kanal 2 angewählt ist
K3	=1, wenn Kanal 3 angewählt ist
FE	=1, wenn "Fehler bei Eingabe" ; Fehler bei Eingabe tritt ein, wenn im 8-Kanal-Kode ein Zeichen mit einer geraden Zahl von Lochungen eingelesen werden soll (Paritätskontrolle); ausgenommen sind "Irrung Zeichen" und das Transportloch im Vorspann, die bei der Lochbandeingabe übersprungen werden (Originaldokumentation 4.3.2.2, Abb. 16)
FÜ	=1, wenn ein Überlauf auftritt. Ein Halt wegen Überlauf setzt weder US noch BS; angezeigt wird nur FÜ (Lampe FÜ).

Flag-Name	Beschreibung
BS	=1, wenn ein programmierter "bedingter Stopp" wirksam wird, d. h. wenn der Befehl abgearbeitet wird und der Auswahlschalter BS (Selektor, gleichnamig, s. Bedienpult-Dokument) eingerastet ist. Ist der Schalter nicht eingerastet, bleibt BS=0 und der Befehl hat keine Wirkung.
US	=1, wenn der Rechner einen "unbedingten Stopp" Befehl ausführt (immer, sobald der Befehl abgearbeitet wird; der Adreßteil des Befehls kann dem Adreßregister entnommen werden).
V1	Vergleicher 1, =1, wenn gesetzt
V2	Vergleicher 2, =1, wenn gesetzt, V1+V2 sind die 2 Bit des (Pseudo)Vergleichers V3, V3 true ist Sprachgebrauch für V1==1 und V2==1
H	=1, wenn Schalter H-R auf Stellung "H" (Halt) steht; =0 bei Stellung "R" (Rechnen)
SP	=1, wenn Selektor-Schalter "SP" gesetzt ist; Bedingung für "Bedingter Sprung mit Rückkehrabsicht"

Anzeige des Arbeitszustands (Bedienungsanleitung 2.3.2 bis 2.3.5): Die Lampen H, R, US, BS, FÜ, FE, E und A zeigen das Rechnen oder den Grund für die Unterbrechung des Befehlsablaufs an. Ein Anhalten erfolgt grundsätzlich nach der Phase III. Bei gesetztem Schalter H leuchtet die Lampe H; im Zustand "Rechnen" darf von diesen Lampen nur R leuchten. US leuchtet beim Befehl "Unbedingter Stopp" (der Adreßteil kann dem Adreßregister entnommen werden), BS beim programmierten "Bedingten Stopp" bei gesetztem Schalter BS, FÜ bei Überlauf (Unterbrechung nur bei Einzelbefehlen ohne Verschiebung, wenn der nachfolgende Befehl kein Überlaufsprung ist), FE bei einem durch die Paritätskontrolle ermittelten Eingabefehler im 8-Kanal-Kode. Die Lampen K0 ("T-SW"), K1, K2, K3 zeigen den angewählten Kanal, bei K0 unterscheiden T und SW zwischen Tastatur und Schreibwerk. Die Lampen V(1-2) zeigen den Vergleicherstand zur Zeit des Setzens: Lampe 1 = Vergleicher 1, Lampe 2 = Vergleicher 2, beide = Vergleicher 3. Die untere Lampenreihe zeigt das Befehlsregister in Triaden: vier Triaden Adreßteil (Gewichte 1000, 100, 10, 1 oktal), drei Triaden Operationsteil (B, A, Z). Die Lampe B ist die Betriebsanzeige, die Lampe N zeigt an, dass die Netzstromversorgung anliegt (Lampenfeld: Abb. 6 der Bedienungsanleitung, Abbildung in `C8205-Interpreter-Bedienpult_mit_Schaltern.md`).

Register und Begriffe des Befehlsablaufs

Register sind:

Kürzel	Bezeichnung	Breite (dezimal)	Anmerkung
AC	Akkumulator	33 Bit	
V1 / V2	Vergleicher	1 Bit	dieselben Elemente wie die Flags V1 und V2 (Vergleicher sind Register und Flags zugleich)
OR	Operationsregister	9 Bit	OR ist identisch mit BAZ
BZ	Befehlszähler	12 Bit	
AR	Adreßregister	12 Bit	
BR	Befehlsregister	21 Bit	BR:=a||OR (a verkettet mit OR)

Weitere Begriffe bei der Beschreibung des Befehlsablaufes sind:

Kürzel	Bezeichnung	Breite (dezimal)	Anmerkung
a	Adresse	12 Bit	Adresse im Adreßteil eines Befehlswortes
a[1-4]	Adreß-Oktal-Ziffer	3 Bit	a:=a1 a2 a3 a4
	Spur	7 Bit	Bitbreite von Spur-Nr passt exakt zur Trommelgeometrie: 7 Bit → 128 Spuren
	Sektor (auch Zelle)	5 Bit	Bitbreite von Sektor-Nr passt exakt zur Trommelgeometrie: 5 Bit → 32 Sektoren
	Speicherwort	33 Bit	
	Maschinenwort	33 Bit	
	Speicher-Trommel	4096 Maschinenworte	

Maschinenwort

- Das interne Maschinenwort besteht aus **33 Bit**.
- Es kann gleichermaßen zur Darstellung von **Zahlen, Befehlen oder Alpha-Zeichen** dienen; die Art der Interpretation wird durch das Programm bestimmt. Verschiedenartige Interpretationen innerhalb eines Wortes sind möglich.
- Die 33 Bits werden von links nach rechts als **$z_0, z_1, \ldots, z_{32}$** bezeichnet. Jedes Bit kann mit 0 oder 1 besetzt sein (2^{33} mögliche Wörter). In jeder Speicherzelle kann ein Wort untergebracht werden.

Die 33 Bit des Maschinenwortes lassen sich in **11 Dreiergruppen (Triaden)** t_{10} bis t_0 aufteilen (33 = 11 × 3 Bit, ohne Rest). Jede Triade entspricht genau einer Oktalziffer.

Befehlswort

Das 33-Bit-Maschinenwort enthält in den Bit z_{12} – z_{32} das Befehlswort. Es gliedert sich (in dieser Interpretation) wie folgt:

Feld	Bits (z-Index)	Breite
"beliebig" (Original-Bezeichnung)	z_0 – z_{11}	12 Bit
Maschinenbefehl, davon:	z_{12} – z_{32}	21 Bit
Adresse, davon:	z_{12} – z_{23}	12 Bit
— Spur-Nr (= Bahn, Synonym)	z_{12} – z_{18}	7 Bit
— Sektor-Nr	z_{19} – z_{23}	5 Bit
Operation, davon:	z_{24} – z_{32}	9 Bit
— B (Befehlstyp)	z_{24} – z_{26}	3 Bit
— A (Operationsart)	z_{27} – z_{29}	3 Bit
— Z (Operationszusatz), davon:	z_{30} – z_{32}	3 Bit
—— R (Resultatbehandlung) ; ACHTUNG: Nicht mit dem R-Befehl verwechseln	z_{30} – z_{31}	2 Bit
—— L (Löschbit)	z_{32}	1 Bit

Adresseinheit (1°): Die Originaldokumentation schreibt 1° für eine Adresseinheit, also die nächste Speicherzelle (z. B. ⟨BZ⟩ + 1° beim Sprungbefehl). In Registern, die nur Adressen enthalten (BZ, AR) und bei Rechnungen mit Adressen ist 1° = 1. Steht eine Adresse im Maschinenwort, so liegt sie in z_{12}–z_{23} und es folgen die 9 Bit der Operation; in Oktalschreibweise (11 Oktalstellen) lautet das Wort `0000` $a_1a_2a_3a_4$ `BAZ`. Eine im Speicher abgelegte Adresseinheit ist daher im ganzen Maschinenwort nicht 1, sondern 1° = `00000001000` (oktal, kurz `01000`, wie `adreinheit dw 01000` in der Assemblerdokumentation). Der Interpreter rechnet intern mit der Adresse allein (12 Bit), dort ist 1° = 1.

Das Feld "Operation" (z_{24} – z_{32}) bezeichnet die Bits des Maschinenbefehls, die festlegen, welche konkrete Durchführung (welcher bestimmte Befehl) gemeint ist. "Operation" im Sinne einer elektronischen Reaktion ist dagegen die Durchführung dieses Befehls in der Elektronik des Rechners: die Ausführung eines Maschinenbefehls (kurz: Befehl, auch Befehlswort genannt). Ein Maschinenbefehl (aus dem Speicher kommend) steht während seiner Ausführung im BR. Er besteht aus der Adresse ADR (auch a genannt) und dem Operationsteil BAZ (auch OP genannt). Das Operationsregister OR ist eine elektronische Baueinheit des Rechners; im OR steht der Operationsteil. Der Maschinenbefehl wird in der Regel aus dem Trommelspeicher geholt. Dort liegt er als Teil (21 Bit) eines Maschinenwortes (33 Bit) auf einer Befehlsadresse BADR. ADR ist dabei jede beliebige Adresse, BADR die Adresse eines Befehls; im Sprachgebrauch wird das oft nicht unterschieden.

Die spitzen Klammern <> angewendet bei Registern und Speicheradressen haben die Bedeutung "Inhalt von". Bei Registern ist das auch die einzige Lesart, beim Speicher aber macht es den Unterschied; <1000> und 1000, das sind der Inhalt der Adresse 1000 und die Adresse 1000 selbst.